DESPLIEGUE DE APLICACIONES · 2º DAW

Git, ramas y GitHub Actions

Todo el tema explicado con calma, sin prisas y con los ejercicios resueltos paso a paso.

1 Git desde cero · las actividades iniciales

2 Ramas y tags · el examen práctico resuelto

3 ¿Qué narices es CI/CD?

4 GitHub Actions · workflows en YAML

5 Custom Actions · crear tus propias actions

★ Chuleta final para repasar antes del examen

Cómo usar estos apuntes: lee las cajas verdes **EN CRISTIANO** para entender la idea, y las naranjas **OJO EXAMEN** para lo que más puede caer. Los comandos, pruébalos en tu terminal: Git se aprende tecleando.

PARTE 1

Git desde cero

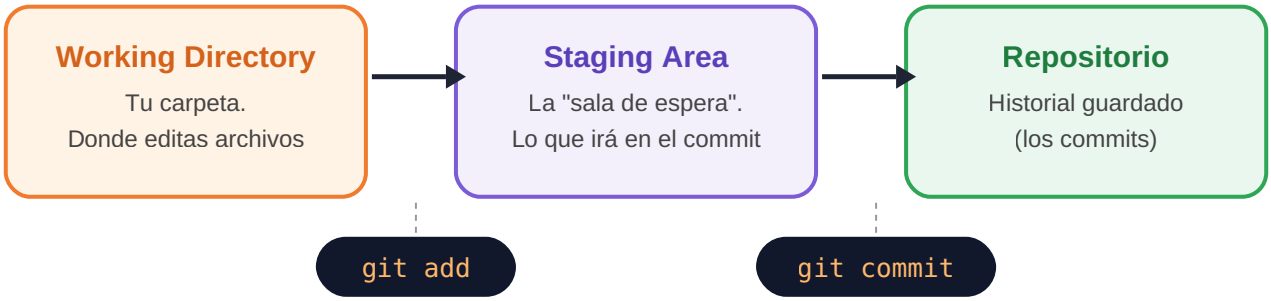
Lo básico que hay que dominar antes de nada: guardar versiones de tu código.

EN CRISTIANO

Git es como el **historial de guardado de un videojuego**. Cada vez que haces un *commit* creas un "punto de guardado" de tu proyecto. Si la lías, vuelves a un punto anterior. Y GitHub es simplemente la **nube** donde subes esos guardados.

Las 3 zonas de Git (lo más importante)

Todo en Git gira alrededor de esto. Un archivo viaja por tres sitios:



EJEMPLO DE LA VIDA REAL

Piensa en hacer la maleta: tu habitación es el **working directory**, la ropa que vas metiendo en la maleta es el **stage** (`git add`) y cuando cierras la maleta y la facturas, eso es el **commit**.

Comandos básicos

Comando	Qué hace
`git init`	Convierte la carpeta actual en un repositorio (crea la carpeta oculta `.git`).
`git status`	Te dice en qué estado está todo: qué archivos han cambiado, cuáles están en stage... **Úsalo siempre.**
`git add fichero`	Mete el fichero en el stage. `git add .` mete todo.
`git commit -m "msg"`	Crea el punto de guardado con un mensaje.
`git log`	Muestra el historial de commits.
`ls -la`	(No es de Git) Lista archivos **incluidos los ocultos**, así ves el `.git` .

Deshacer cosas: `git reset`

Para "volver atrás" un commit se usa `git reset`. `HEAD~1` significa "el commit anterior al actual". Lo que cambia es **qué pasa con tus cambios**:

Comando	¿Se borra el commit?	¿Qué pasa con los cambios?
`--soft HEAD~1`	Sí	Se quedan en el **stage**, listos para volver a hacer commit.
`--mixed HEAD~1` (el de por defecto)	Sí	Se quedan en tus archivos pero **fuera** del stage.
`--hard HEAD~1`	Sí	**Desaparecen del todo.** Ni stage ni archivos. Cuidado.

OJO EXAMEN

Truco para acordarte: **soft** = suave, no toca nada, solo quita el commit. **hard** = duro, lo arrasa todo. Si te piden "deshacer pero que se quede en el stage" → `--soft`. Si dicen "que no exista ni en el stage ni en los ficheros" → `--hard`.

Configurar Git (se hace una vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
git config --global core.editor "nano"      # o "code --wait" para VS Code
git config --list                              # ver toda tu configuración
```

EN CRISTIANO

Cada commit lleva "firma" (quién lo hizo). Por eso Git necesita saber tu nombre y tu email. `--global` significa "para todos mis repos de este ordenador".

Actividades iniciales · resueltas

1 Crear repo y ver contenido (también lo oculto)

```
mkdir activitatsPrimeraClaseDAW
cd activitatsPrimeraClaseDAW
git init
ls -la          # la "a" muestra ocultos → verás la carpeta .git
```

2 Crear README.md, mirar estado, añadir al stage, mirar otra vez

```
nano README.md     # pega el texto del enunciado y guarda
git status       # sale en ROJO: "Untracked files"
git add README.md
git status       # ahora en VERDE: "Changes to be committed"
```

3 Commit con mensaje concreto

```
git commit -m "Afegit fitxer Readme"
```

4 Ver el log en una línea con hash, autor, fecha y hora

```
git log                          # versión larga
git log --oneline                # una línea: hash corto + mensaje
git log --pretty=format:"%h | %an | %ad" --date=format:"%d/%m/%Y %H:%M"
```

Los "códigos": `%h` hash corto · `%H` hash completo · `%an` autor · `%ad` fecha · `%s` mensaje.

5 Meter un error, hacer commit y deshacerlo dejándolo en el stage

```
nano README.md                   # escribe cualquier error
git add README.md && git commit -m "Error"
git reset --soft HEAD~1          # commit fuera, cambio sigue en stage
git status                       # compruébalo: sale en verde
```

6 Otro error, commit y borrarlo del todo

```
git commit -am "Otro error"      # -a añade lo modificado y hace commit
git reset --hard HEAD~1          # adiós commit, stage y cambios
```

7 Configurar nombre, email y editor

Está en la página anterior: `git config --global user.name / user.email / core.editor`.

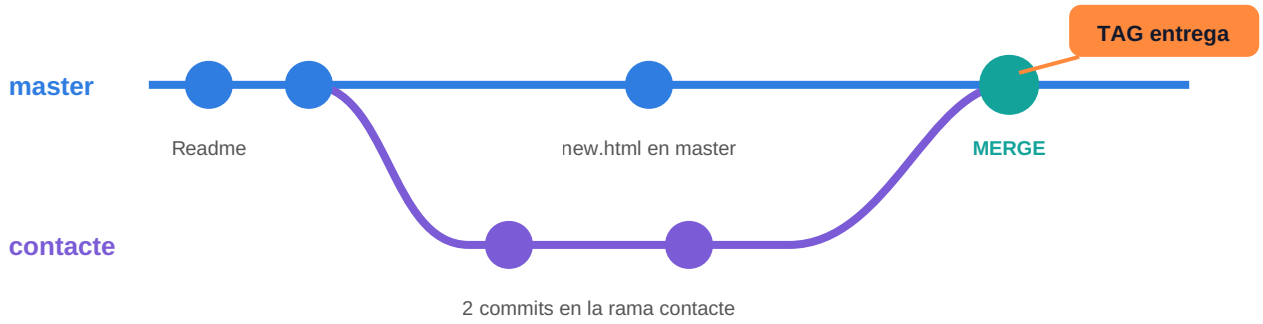
PARTE 2

Ramas y tags

Trabajar en algo nuevo sin romper lo que ya funciona. Y marcar versiones.

EN CRISTIANO

Una **rama** es una "línea temporal paralela". Te creas una copia del proyecto, haces tus experimentos ahí, y cuando todo funciona la **fusionas** (merge) con la rama principal. Si sale mal, la borras y no ha pasado nada.



Así queda exactamente el historial del examen práctico.

Comandos de ramas

Comando	Qué hace
`git branch`	Lista las ramas. La que tiene * es en la que estás.
`git branch nombre`	Crea una rama (pero NO te mueve a ella).
`git switch nombre` `git checkout nombre`	Te mueve a esa rama (los dos hacen lo mismo).
`git switch -c nombre` `git checkout -b nombre`	Crea la rama **y** te mueve a ella. Dos en uno.
`git merge nombre`	Trae los cambios de *nombre* a la rama donde estás ahora.
`git branch -d nombre`	Borra la rama (solo si ya está fusionada). `-D` fuerza.
`git log --oneline --graph --all`	Dibuja el árbol de ramas en la terminal. Muy útil.

OJO EXAMEN · EL MERGE

El merge se hace **desde la rama que recibe**. Si quieres meter `contacte` dentro de `master`: primero `git switch master`, luego `git merge contacte`. Al revés lo harías mal.

Tags: ponerle nombre a un commit

Un **tag** es una etiqueta fija en un commit concreto, tipo "v1.0" o "entrega-final". A diferencia de una rama, **no se mueve**.

Tag ligero

Solo un nombre.

```
git tag v1.0
```

Tag anotado ★

Guarda autor, fecha y mensaje. **Es el que piden.**

```
git tag -a v1.0 -m "Versión 1"
```

Comando	Qué hace
`git tag`	Lista los tags.
`git show nombre-tag`	Muestra la info del tag y su commit.
`git push origin nombre-tag`	Sube **ese** tag a GitHub.
`git push --tags`	Sube **todos** los tags.

OJO EXAMEN · PREGUNTA TRAMPA

Un `git push` normal **NO sube los tags**. Hay que subirlos aparte con `git push origin <tag>` o `git push --tags`. El enunciado te lo recuerda a propósito.

Remotos (GitHub)

```
git remote add origin https://github.com/usuario/repo.git  # conectar con GitHub
git remote -v                                              # comprobar que está
git push -u origin master                                  # subir (la 1ª vez con -u)
```

QUÉ ES "ORIGIN"

Es solo el **apodo** del repo de GitHub, para no escribir la URL cada vez. `-u` lo deja vinculado, así la próxima vez basta con `git push`.

PARTE 2 · PRÁCTICA

Examen de ramas y tags, resuelto

Paso a paso. Intenta hacerlo tú primero y mira esto si te atascas.

ANTES DE EMPEZAR

Según cómo tengas Git configurado, la rama principal puede llamarse `master` o `main`. Mira cuál sale con `git branch`. Si quieres que siempre sea master: `git config --global init.defaultBranch master`.

0 Crear el repositorio

```
cd ~                            # ~ = tu home
mkdir -p git/web && cd git/web
git init
echo "# Mi web" > Readme.md
git add Readme.md && git commit -m "Readme inicial"
```

1 Comprobar rama y crear "contacte"

```
git status                    # "On branch master, nothing to commit" = limpio
git switch -c contacte        # crea la rama y se mueve
```

2 Dos commits (mínimo) en la rama

```
nano index.html               # pega la <section id="contacte"> con el h2
git add index.html && git commit -m "Añade sección contacte"
nano index.html               # añade el <p> con el enlace mailto
git commit -am "Añade enlace de correo"
git rev-list --count HEAD     # cuántos commits hay → 3
git log --oneline             # o verlos y contarlos
```

3 Volver a master y hacer un cambio independiente

```
git switch master
nano new.html                 # pega la línea <meta name="description"...>
git add new.html && git commit -m "Añade new.html"
```

Fíjate: si haces `ls` en master **no verás index.html**. Está solo en la otra rama.

4 Fusionar contacte en master

```
git merge contacte           # estando en master. Se abre el editor → guarda y sal
git log --oneline --graph --all   # mira el dibujo del merge
```

5 Ver el contenido de cada rama

```
git ls-tree -r --name-only master     # Readme.md, index.html, new.html
git ls-tree -r --name-only contacte   # Readme.md, index.html (sin new.html)
```

Alternativa sencilla: `git switch contacte` + `ls`, y luego `git switch master` + `ls`.

6 Tag anotado de la entrega (en master)

```
git tag -a entrega-fabian -m "Entrega de hoy"
git tag                        # comprobar que existe
```

El **push del tag** va después del paso 8, cuando ya tienes remoto.

7 Borrar la rama ya fusionada

```
git branch -d contacte
```

8 Subir a GitHub (y luego el tag)

```
# antes: crea el repo VACÍO en github.com (sin README)
git remote add origin https://github.com/TU_USUARIO/web.git
git push -u origin master
git push origin entrega-fabian     # ← vuelta al punto 6: subir el tag
```

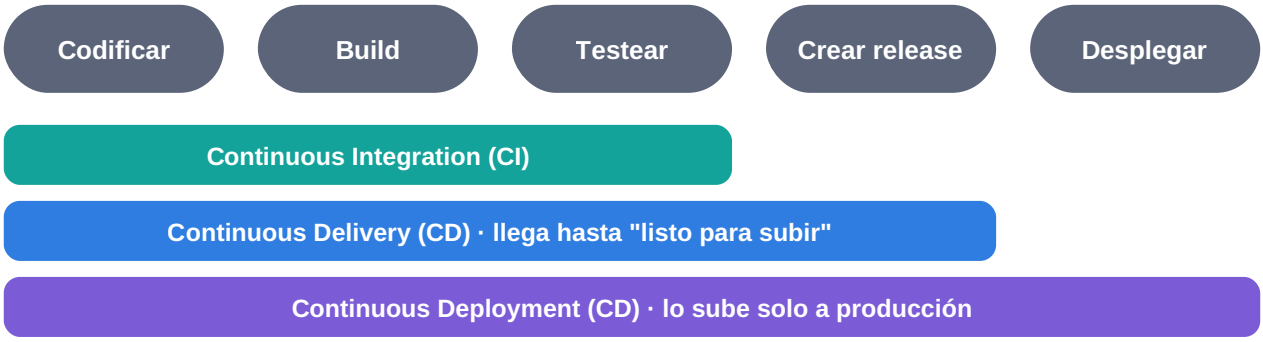
PARTE 3

¿Qué es CI/CD?

La teoría que hay detrás de GitHub Actions. Tres conceptos y ya.

EN CRISTIANO

CI/CD es **automatizar** lo aburrido. En vez de que una persona compile, pruebe y suba la web a mano cada vez que alguien cambia algo, lo hace un robot automáticamente cada vez que haces `push`.



CI **Continuous Integration · Integración continua**

Los devs suben código **muchas veces al día** a un repo compartido. Cada subida se comprueba con **tests automáticos** para pillar los errores pronto.

CD **Continuous Delivery · Entrega continua**

Extiende la CI: el código siempre queda **listo para subir a producción**, pero el último botón de "desplegar" lo pulsa **una persona**.

CD **Continuous Deployment · Despliegue continuo**

Lo mismo pero **sin humano**: si pasa todos los tests, se sube solo a producción. Requiere mucha confianza en los tests.

OJO EXAMEN · LA DIFERENCIA CLAVE

Las dos "CD" se confunden siempre. **Delivery** = listo para desplegar, **decide un humano**. **Deployment** = se despliega **automáticamente**, sin intervención humana.

Herramientas para hacer CI/CD

GitHub Actions es la de GitHub (la que damos). Alternativas que pueden nombrar: **GitLab CI/CD**, **Jenkins** (el del mayordomo) y **Travis CI**.

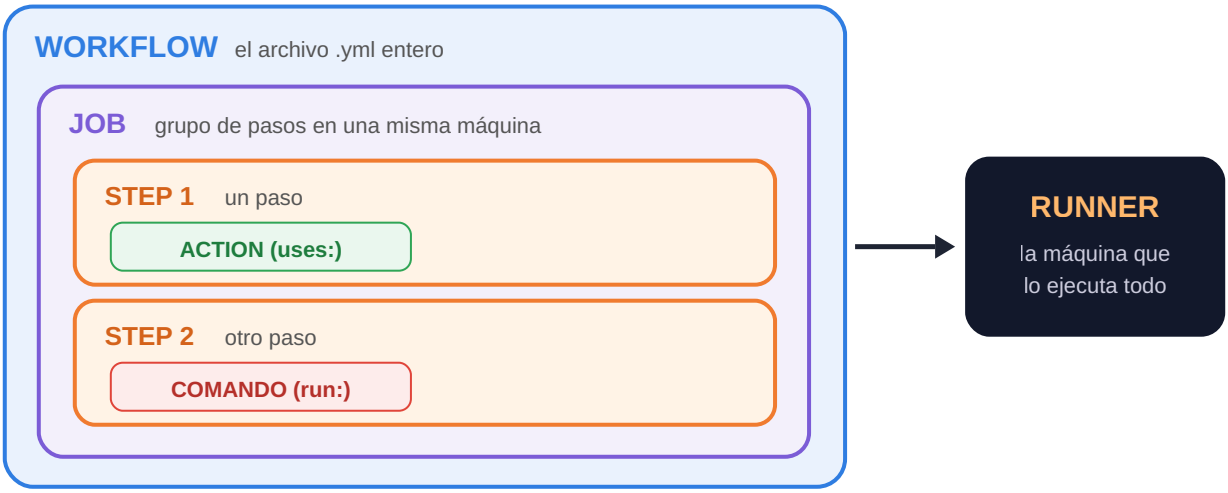
PARTE 4

GitHub Actions

El "robot" de GitHub que ejecuta tareas automáticas cuando pasa algo en tu repo.

Las piezas, de grande a pequeña

Funciona como las **muñecas rusas**: una cosa dentro de otra.



Pieza	Qué es	Comparación
Workflow	El proceso automático completo. Uno o varios jobs. Se activa con un **evento** (push, PR...).	La receta entera
Job	Conjunto de steps que se ejecutan en el **mismo runner**. Por defecto los jobs van **en paralelo**.	Un plato de la receta
Step	Una tarea individual dentro de un job. Puede ser una action o un comando de shell.	"Pela las patatas"
Action	Bloque reutilizable ya hecho (por GitHub, por otros o por ti). La pieza más pequeña.	Un electrodoméstico
Runner	El servidor que ejecuta los jobs y devuelve el resultado.	El cocinero

Tipos de runner

GitHub-hosted (en la nube)

- + Cero mantenimiento, todo hecho
- + 3 sistemas: Windows, macOS y Ubuntu
- − Se paga por uso (gratis 2000 min/mes)
- − Para configs muy raras puede no servir

Self-hosted (en tu máquina)

- + Accede a tu red local y tus recursos
- + Total flexibilidad de configuración
- + No pagas por usarlo
- − Lo tienes que configurar y mantener tú

Anatomía de un workflow

Se escriben en **YAML** y se guardan en la carpeta `.github/workflows/` de tu repo. Si no están ahí, GitHub no los ve.

```yaml
name: primer-ejemplo              # ① nombre (opcional) que sale en la pestaña Actions
on: [push]                        # ② CUÁNDO se ejecuta (obligatorio)
jobs:                             # ③ aquí van los trabajos
  check-bats-version:             # ④ id del job (lo eliges tú)
    runs-on: ubuntu-latest         # ⑤ en qué máquina (obligatorio)
    steps:                         # ⑥ pasos, se ejecutan EN ORDEN
      - uses: actions/checkout@v2   # usa una action: descarga tu código
      - uses: actions/setup-node@v1 # usa una action: instala Node
      - run: npm install -g bats    # ejecuta un comando
      - run: bats -v                # ejecuta otro comando
```

OJO EXAMEN · USES vs RUN

`uses` = usar una **action** ya hecha (algo del tipo `usuario/repo@versión`). `run` = ejecutar un **comando** de terminal. Esta diferencia cae seguro.

YAML: LA REGLA DE ORO

En YAML la **indentación (los espacios) lo es todo**. Usa siempre 2 espacios, nunca tabuladores. Un espacio mal puesto y el workflow no funciona.

`on` · cuándo se dispara

```yaml
on: push                          # un evento
on: [push, pull_request]          # varios eventos

on:                                # con filtros: solo en la rama main
  push:
    branches: [main]
    tags: [v1]                     # o al subir el tag v1
  pull_request:
    branches: [main]
```

`needs` · ordenar jobs

Los jobs van en paralelo por defecto. Con `needs` dices "espera a que termine este otro".

```yaml
jobs:
  job1:
  job2:
    needs: job1                   # job2 espera a job1
  job3:
    needs: [job1, job2]           # job3 espera a los dos
    if: always()                  # ...y se ejecuta AUNQUE hayan fallado
```

OJO EXAMEN

Si un job falla, los que dependen de él (`needs`) **no se ejecutan**. Excepto si llevan `if: always()`, que se ejecutan pase lo que pase.

`runs-on` · la máquina

Obligatorio. Elige el runner: `ubuntu-latest`, `windows-latest`, `macos-latest`... o uno **self-hosted** tuyo.

`env` · variables de entorno

Se pueden poner a nivel de **workflow**, de **job** o de **step**.

```yaml
env:
  SERVER: production              # nivel workflow → vale para todo
jobs:
  job1:
    env:
      FIRST_NAME: Mona            # nivel job → solo para este job
    steps:
      - run: echo $SERVER $FIRST_NAME
```

> **OJO EXAMEN**
> Si la misma variable está definida en varios niveles, **gana la más específica** (step > job > workflow). Como en CSS: lo más concreto manda.

`steps`: `uses` y `run` a fondo

```yaml
steps:
  - name: Descargar código           # nombre bonito del paso (opcional)
    uses: actions/checkout@v2        # action de terceros, versión v2
  - name: Mi action propia
    uses: ./.github/actions/my-action # action que está en TU repo
  - name: Instalar y compilar
    run: |                           # la barra | = varios comandos seguidos
      npm ci
      npm run build
  - name: Limpiar
    run: rm -rf *
    working-directory: ./temp         # en qué carpeta ejecutarlo
```

Hay miles de actions ya hechas en el **GitHub Marketplace**.

`strategy.matrix` · un job, muchas combinaciones

> **EN CRISTIANO**
> La matriz sirve para decir "ejecuta este mismo job varias veces, cambiando un valor". Típico: probar tu app con varias versiones de Node a la vez.

```yaml
strategy:
  matrix:
    os: [ubuntu-22.04, ubuntu-24.04]  # 2 opciones
    node: [6, 8, 10]                   # 3 opciones
runs-on: ${{ matrix.os }}               # así se usa el valor
steps:
  - uses: actions/setup-node@v1
    with:
      node-version: ${{ matrix.node }}
```

OJO EXAMEN · CUÁNTOS JOBS SE CREAN

Se **multiplica**: 2 sistemas × 3 versiones de Node = **6 jobs**. Si solo hubiera `node: [6, 8, 10]` serían 3. Con `include` defines combinaciones concretas a mano (con sus propias variables).

`${{ ... }}` es la forma de "leer" un valor dentro del YAML (de la matriz, de inputs, de outputs de otros pasos...).

PARTE 5

Custom GitHub Actions

Crear tus propias actions en vez de usar solo las de otros.

EN CRISTIANO

Hasta ahora usábamos actions de otros (`actions/checkout` ...). Una **custom action** es una que programas tú, para reutilizarla en tus workflows. Recuerda la jerarquía: **Workflow → Job → Step → Action**. La action vive dentro de un step.

Dos tipos posibles

Docker container action

- + Más consistente y fiable
- + No te preocupas por dependencias
- − **Solo en runners Linux**
- − Más lentas

JavaScript action

- + Código más simple
- + **Ejecución más rápida**
- + Funciona en todos los runners...
- − ...solo si está en JavaScript puro

OJO EXAMEN

Docker = **fiable pero lento y solo Linux**. JavaScript = **rápido y en cualquier sistema**. Pregunta típica de test.

El archivo `action.yml`

Toda action necesita un archivo de **metadatos** llamado `action.yml` (o `.yaml`). Es como su "ficha técnica": qué recibe, qué devuelve y qué ejecuta.

```yaml
name: 'Hello World'                          # OBLIGATORIO
description: 'Greet someone and record the time'  # OBLIGATORIO
inputs:                                      # lo que RECIBE (opcional)
  who-to-greet:
    description: 'Who to greet'
    required: true
    default: 'World'                        # el único atributo opcional
outputs:                                     # lo que DEVUELVE (opcional)
  time:
    description: 'The time we greeted you'
runs:                                        # CÓMO se ejecuta (OBLIGATORIO)
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
```

Campo	¿Obligatorio?	Para qué sirve
`name`	SÍ	Nombre que se ve en la pestaña Actions.
`author`	NO	Quién la ha hecho.
`description`	SÍ	Descripción breve.
`inputs`	NO	Parámetros de entrada. Dentro, todo obligatorio **menos** `default`.
`outputs`	NO	Datos que la action devuelve. Dentro, todo obligatorio.
`runs`	SÍ	Cómo se ejecuta. Cambia si es JavaScript o Docker.
`branding`	NO	Icono y color si la publicas en el Marketplace.

DATO CURIOSO QUE PUEDE CAER

Por cada input, GitHub crea una **variable de entorno** llamada `INPUT_<NOMBRE>`, en mayúsculas y cambiando espacios por `_`. Ejemplo: el input `name` → `INPUT_NAME`.

`runs` según el tipo

Versión JavaScript

```yaml
runs:
  using: 'node12'   # OBLIG.
  pre: 'setup.js'
  main: 'index.js'  # OBLIG.
  post: 'cleanup.js'
```

- `using`: con qué se ejecuta
- `main`: el código principal
- `pre`: script **antes** de main (condicional con `pre-if`)
- `post`: script **después** (condicional con `post-if`)

Versión Docker

```yaml
runs:
  using: 'docker'    # OBLIG.
  image: 'Dockerfile' # OBLIG.
  args:
    - 'bzz'
  entrypoint: 'main.sh'
```

- `image`: un Dockerfile local o una imagen de Docker Hub
- `args`: entradas para el contenedor
- `entrypoint`: sustituye al ENTRYPOINT del Dockerfile

Ejemplo JavaScript · los 6 pasos

1 Crear `action.yml`

En `.github/actions/hello-world-javascript-action/action.yml`, con `runs: using: 'node12'` y `main: 'index.js'`.

2 Crear proyecto Node en esa carpeta

```
npm init -y
```

3 Instalar las librerías oficiales

```
npm install @actions/core      # leer inputs, escribir outputs, marcar fallos
npm install @actions/github    # info del repo y del evento
```

4 Escribir `index.js`

```
const core = require('@actions/core');
const github = require('@actions/github');
try {
  const name = core.getInput('who-to-greet');   // lee el input
  console.log(`Hello ${name}!`);
  const time = (new Date()).toTimeString();
  core.setOutput("time", time);                 // devuelve el output
} catch (error) {
  core.setFailed(error.message);                // marca el step como fallido
}
```

5 Compilar con ncc

```
npm i -g @vercel/ncc
ncc build index.js --license licenses.txt
# y en action.yml cambias:  main: 'dist/index.js'
```

¿Por qué? Para meterlo todo en un único archivo y **no tener que subir la carpeta** `node_modules` al repo (que pesa una barbaridad).

6 Probarla en un workflow

```
on: [push]
jobs:
  hello_world_job:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2           # 1º SIEMPRE descargar el repo
      - uses: ./                            # tu action (ruta a su carpeta)
        id: hello                           # id para poder leer su output
        with:                               # with = pasarle los inputs
          who-to-greet: 'Mona the Octocat'
      - run: echo "The time was ${{ steps.hello.outputs.time }}"
```

OJO EXAMEN · 3 DETALLES CLAVE

1. Para usar una action de tu propio repo, antes hay que hacer `actions/checkout`, si no, el runner no tiene tu código.

2. Los inputs se pasan con `with:`.

3. Para leer el output de un step, el step necesita `id`, y se lee con `${{ steps.<id>.outputs.<nombre> }}`.

Ejemplo Docker · 3 pasos

1 Dockerfile + script

```dockerfile
# Dockerfile
FROM alpine:3.10
COPY entrypoint.sh /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

```sh
# entrypoint.sh
#!/bin/sh -l
echo "Hello $1"
time=$(date)
echo "::set-output name=time::$time"
```

`FROM` = imagen base · `COPY` = copia tu script dentro · `ENTRYPOINT` = lo que se ejecuta al arrancar. `$1` es el primer argumento (el input que llega por `args`).

2 `action.yml` con `using: 'docker'` e `image: 'Dockerfile'`

Igual que el de la página de `action.yml`, con `args: - ${{ inputs.who-to-greet }}`.

3 Probarla en un workflow

Exactamente el mismo workflow que en el ejemplo JavaScript. Al workflow le da igual cómo esté hecha la action por dentro.

> **NOTA: LAS DIAPOS ESTÁN UN POCO ANTIGUAS**
>
> En la vida real hoy se usa `node20` en vez de `node12`, `checkout@v4` y, en vez de `::set-output`, se escribe `echo "time=$time" >> $GITHUB_OUTPUT`. Para el examen, **responde como en las diapositivas**, pero que sepas que existe esto.

REPASO FINAL

Chuleta para la noche antes

Si solo te da tiempo a leer una página, que sea esta.

Git básico

- `git init` · `git status`
- `git add .` · `git commit -m ""`
- `git log --oneline`
- `reset --soft` → queda en stage
- `reset --hard` → se borra todo
- `git config --global user.name`

Ramas y tags

- `git switch -c rama` (crear + ir)
- merge: ve a master → `git merge rama`
- `git branch -d rama` (borrar)
- `git tag -a nombre -m ""` (anotado)
- `git push origin tag` ← push normal NO sube tags
- `git remote add origin URL`

CI/CD

- **CI**: integrar a menudo + tests automáticos
- **Delivery**: listo para subir, decide un humano
- **Deployment**: se sube solo a producción
- Alternativas: GitLab CI, Jenkins, Travis

GitHub Actions

- Workflow → Job → Step → Action · Runner ejecuta
- Archivos en `.github/workflows/`
- Obligatorios: `on` y `runs-on`
- Jobs en paralelo salvo `needs`
- `uses` = action · `run` = comando
- env: gana la más específica · matrix multiplica

Custom Actions

- Tipos: **Docker** (fiable, lenta, solo Linux) y **JavaScript** (rápida, cualquier SO)
- Metadatos en `action.yml`
- Obligatorios: `name`, `description`, `runs`
- Input → variable `INPUT_NOMBRE`
- JS: `using` + `main` · Docker: `using` + `image`
- `ncc` compila para no subir `node_modules`
- Inputs con `with:`, outputs con `steps.id.outputs.x`

Autotest rápido (tapa las respuestas)

Pregunta	Respuesta
¿Qué reset deja los cambios en el stage?	`git reset --soft HEAD~1`
¿Cómo subo un tag a GitHub?	`git push origin nombre-tag`
¿Diferencia entre Delivery y Deployment?	En Deployment no hay intervención humana.
¿Dónde van los workflows?	`.github/workflows/`
Por defecto, ¿los jobs van en serie o en paralelo?	En paralelo.
Matriz con 2 SO y 3 versiones de Node, ¿cuántos jobs?	6
¿Qué custom action solo funciona en Linux?	La de Docker.
¿Qué campos son obligatorios en `action.yml`?	`name`, `description` y `runs`
¿Para qué sirve `ncc`?	Compilar en un archivo y no subir `node_modules`.
Runner gratis en la nube, ¿cuántos minutos?	2000 min/mes